

Strategic Sourcing Intelligence & RAG Contract Analytics Platform

Exhaustive Blueprint: Docling Parsing Engine, DB Schemas, Tool Rationale & Anti-Hallucination

Document Parser Engine: IBM Docling (DocumentConverter)	Database & Vector Store: PostgreSQL 16 + pgvector
LLM & Embedding Engine: Gemini 3.8 Flash & text-embedding-004	Container Infrastructure: Docker Compose 3-Tier Stack
Document Release: October 2026	Target Audience: Engineering Leads, AI Architects & Sourcing VPs

1. Executive Summary & Database Inspection Guide

The **Strategic Sourcing Intelligence Platform** is an enterprise RAG system that automates the benchmarking of incoming retail fashion supplier proposals against historical executed awards and category best-in-class standards. The system uses **IBM Docling (docling.document_converter.DocumentConverter)** as its primary document parsing and layout hierarchy extraction engine, converting complex PDFs, DOCX, and HTML contract files into layout-aware Markdown trees for LLM extraction and vector storage.

1.1 How to Inspect the Live Database (PostgreSQL / pgvector)

The database runs inside container rag-db-1 on mapped host port 5433 (container port 5432). Engineers can connect directly using standard docker exec or psql commands:

```
# 1. List all tables in the live database
docker exec -i rag-db-1 psql -U postgres -d rag_gap -c "\dt"

# 2. Inspect document_chunks table schema & IVFFlat vector index
docker exec -i rag-db-1 psql -U postgres -d rag_gap -c "\d document_chunks"

# 3. Query document_chunks table (chunk types, story_id links, metadata)
docker exec -i rag-db-1 psql -U postgres -d rag_gap -c "SELECT chunk_id, chunk_type, story_id, source_path FROM document_chunks LIMIT 5;"

# 4. Inspect users & authentication sessions tables
docker exec -i rag-db-1 psql -U postgres -d rag_gap -c "\d users; \d sessions"
```

2. Tools Used & Technology Selection Rationale

Layer / Choice	Selected Technology	Alternatives Evaluated	Engineering Rationale & Trade-off Advantage
Document Parser	IBM Docling (DocumentConverter)	PyPDF2, pdfplumber, Unstructured, PyPDFium2	Docling preserves structural layout hierarchies (# headings, section levels, table structures) and exports clean Markdown. Supports PDF, DOCX, HTML, MD, ADOC, and TXT seamlessly.
Backend Framework	FastAPI (Python 3.11)	Flask, Django, Express.js	Asynchronous execution (asyncio), automatic OpenAPI schema validation via Pydantic, high throughput, and native integration with Python AI ecosystem.

Database & Vector Store	PostgreSQL 16 + pgvector	Pinecone, ChromaDB, Weaviate, Qdrant	Eliminates dual-database synchronization issues. Stores relational users, sessions, metadata JSONB, and dense vector embeddings in a single ACID-compliant database with IVFFlat cosine indexing.
Frontend UI	React 18 + Vite + TS	Next.js, Angular, Vue 3	Instant Vite HMR build speeds, component modularity, client-side rendering for complex interactive 4-tier dashboards without SSR overhead.
LLM Engine	Gemini 2.5/3.8 Flash	OpenAI GPT-4o, Claude 3.5 Sonnet	Sub-second inference speed, 1M token context window, structured JSON output mode, and low cost per 1M tokens ideal for bulk contract ingestion.
Containerization	Docker Compose	Bare-metal, VM deployment, K8s	Guarantees identical environment across dev/prod, compiles pgvector extension in-container, enforces healthcheck dependencies (pg_isready).

3. Chunking Schemas & Live Sample Chunks

Flat chunking fails in legal contract analysis because individual clauses lose section context (weight %, section letter, evaluation type). The platform implements a **2-Tier Hierarchical Chunking Model** linked by a deterministic MD5 story_id:

- 1. **story_chunks (Section Level):** Represents macro RFP sections parsed from Docling layout hierarchy. Generated via MD5(doc_title + section_key)[:12]. Used for high-level category retrieval.
- 2. **ac_chunks / criteria_chunks (Clause Level):** Individual clauses containing full criteria text. ID format: {story_id}_{clause_id}. Used for fine-grained gap comparison.

3.1 Live Sample Chunk Objects (JSON & DB Representation)

```
// 1. Sample story_chunk (Section Level Summary from Docling Hierarchy)
{
  "id": "a7b8c9d0e1f2",
  "chunk_type": "story",
  "text": "Section D: Fee Proposal & Allowances (Weight: 35%, Eval: scored) – Extracted 8 clauses",
  "metadata": {
    "story_id": "a7b8c9d0e1f2",
    "story_title": "Section D: Fee Proposal",
    "section_letter": "D",
    "section_name": "Fee Proposal",
    "section_weight": 35,
    "document_type": "baseline_rfp",
    "department": "Mens",
    "product_category": "Bottomwear",
    "material_family": "Denim_Twill"
  }
}

// 2. Sample ac_chunk (Clause Level Item)
{
  "id": "a7b8c9d0e1f2_FIN-D.3",
  "chunk_type": "criteria",
  "text": "[FIN-D.3] Section D (Fee Proposal): Settlement Terms – Net 90 days payment terms with 2.0% DMA",
  "metadata": {
    "ac_id": "FIN-D.3",
    "ac_title": "Settlement Terms & Standard Allowances",
    "clause_type": "commercial",
    "story_id": "a7b8c9d0e1f2",
    "department": "Mens",
    "product_category": "Bottomwear"
  }
}
```

4. LLM System Prompts & Formatting Templates

4.1 Ingestion Prompt (`INGESTION PROMPT`)

You are an expert Strategic Sourcing Classifier. Analyze the Markdown text extracted by Docling below.

1. Assign values matching the controlled 5-part metadata taxonomy:

- document_type: 'baseline_rfp' | 'vendor_bid' | 'historical_award'
- department: 'Mens' | 'Womens' | 'Kids' | 'Unisex' | 'General'
- product_category: 'Topwear' | 'Bottomwear' | 'Footwear' | 'Sports_Activewear' | 'Accessories' | 'General'
- material_family: 'Cotton_Basics' | 'Linen_Natural' | 'Synthetic_Performance' | 'Denim_Twill' | 'General'

2. Parse every clause into an array of objects: { clause_id, clause_title, section_letter, section_name, section_weight, eval_type, clause_text }

Output strictly valid JSON.

4.2 Gap Analysis Prompt (`GAP ANALYSIS PROMPT`)

You are a Strategic Sourcing Intelligence Agent evaluating incoming vendor proposals against historical KB baselines.

1. Benchmark incoming proposal clauses simultaneously against Vendor Self-Precedent & Category Best-in-Class.

2. Generate strict 4-Tier JSON payload:

- Tier 1: Award Status (ACCEPT WITH CONDITIONS | TARGETED RENEGOTIATION | DISQUALIFIED), 100pt Scorecard & Macro Exposure.
- Tier 2: Priority Negotiation Battlecard (Rank top 3-5 deviations with priority, clause_id, clause_title, deviation_summary, impact_badge, cited_anchors).
- Tier 3: Section-Wise Analysis (Clause accordions with insight_statement, quantitative_deltas, evidence).
- Tier 4: One-Click Supplier Counter-Offer Letter text.

5. Anti-Hallucination Engineering Measures

In legal contract analytics, LLM hallucinations (inventing non-existent clauses or false precedents) pose severe risk. The platform implements **6 layered architectural anti-hallucination guardrails**:

- **1. Docling Structural Hierarchy Preservation:** Docling converts complex PDFs into structured Markdown with preserved heading levels (`# Section`, `## Clause`), preventing text truncation and cross-clause merging during LLM ingestion.
- **2. Grounded Dual-Anchor Citation Mandate:** The LLM is strictly prohibited from making generic claims (e.g., 'Violates commercial standard'). Every insight statement and cited anchor must explicitly reference a verified document title retrieved from pgvector (e.g., WMT-AWARD-MENS-ACTIVE-2025-004).
- **3. Raw Evidence Context Window Injection:** The gap analysis engine injects exact raw text strings extracted by Docling directly into prompt context blocks (incoming_bid, vendor_precedent, category_best), forcing the LLM to ground its output in empirical text.
- **4. Strict JSON Schema Validation:** FastAPI enforces Pydantic structural validation on all LLM outputs. Any response failing schema validation is automatically rejected and retried with explicit syntax hints.
- **5. Low Temperature Control:** Inference temperature is set to \$T=0.2\$ for gap analysis and \$T=0.4\$ for chat queries, minimizing creative randomness and maximizing deterministic extraction.
- **6. Source Citation Distance Traceability:** The RAG chat pipeline calculates cosine similarity scores (1.0 - distance) for all retrieved chunks, returning exact source document filenames, section titles, and similarity scores in the response payload.

6. Containerization & Deployment Rationale (Why Docker?)

- **pgvector Extension Compilation:** Compiling C extensions for PostgreSQL (`pgvector`) on bare-metal operating systems (especially Windows/macOS) is error-prone. The `pgvector/pgvector:pg16` container provides a pre-compiled, optimized vector database environment.

- **Multi-Stage Build Efficiency:** The frontend Dockerfile uses multi-stage builds: Node 20 compiles TypeScript and Vite bundles into minified static assets, which are copied to a lightweight Nginx Alpine container (producing a 25MB runtime image).
- **Docling & PyTorch Dependency Isolation:** The backend container encapsulates Python 3.11, Docling dependencies, C build tools (`gcc`, `libpq-dev`), OpenCV libraries (`libgl1`), and PyTorch CPU runtime without polluting the host environment.
- **Service Dependency & Healthchecks:** Docker Compose enforces strict startup ordering using `depends\_on` with `condition: service\_healthy`. The backend waits for PostgreSQL's `pg\_isready` check before executing migrations.
- **Single-Command Enterprise Portability:** The entire multi-service stack (database, API, frontend) spins up seamlessly across any cloud instance or local machine with `docker compose up -d --build`.

7. Comprehensive Architecture Diagrams

7.1 End-to-End Component Topology (with Docling Pipeline)

